

21世纪软件工程专业规划教材

软件工程导论（第6版）

第8章 面向对象设计

分析是提取和整理用户需求，并建立问题域精确模型的过程。**设计**则是把分析阶段得到的需求转变成**符合成本和质量要求的、抽象的系统实现方案**的过程。

从面向对象分析到**面向对象设计（OOD）**，是一个逐渐扩充模型的过程。或者说，面向对象设计就是用面向对象观点建立**求解域**模型的过程。

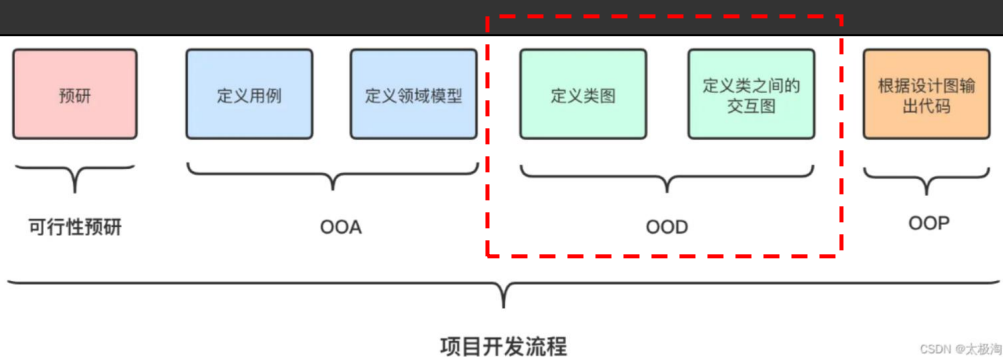
目录

CONTENTS

- | | | | |
|---|-----------|----|-----------|
| 1 | 面向对象设计的准则 | 7 | 设计任务管理子系统 |
| 2 | 启发规则 | 8 | 设计数据管理子系统 |
| 3 | 软件重用 | 9 | 设计类中的服务 |
| 4 | 系统分解 | 10 | 设计关联 |
| 5 | 设计问题域子系统 | 11 | 设计优化 |
| 6 | 设计人机交互系统 | | |

面向对象设计的准则

所谓优秀设计就是权衡了各种因素，从而使得系统在其整个生命周期中的**总开销最小**的设计，对大多数软件系统而言，60%以上的软件费用都用于软件维护，因此优秀软件设计的一个主要特点就是**容易维护**。



① 模块化

对象就是模块，它把数据结构和操作这些数据的方法紧密地结合在一起。

② 抽象：

过程抽象、数据抽象（类）、参数化抽象（C++模版）。

③ 信息隐藏：

通过对象的封装性实现，类分离了接口与实现，支持信息隐藏。

④ 弱耦合：是优秀设计的重要标准

耦合是指一个软件结构内不同模块之间互连的紧密程度。弱耦合：系统中某一部分的变化对其他部分的影响降到最低程度，对象之间的耦合包括**交互耦合** & **继承耦合**

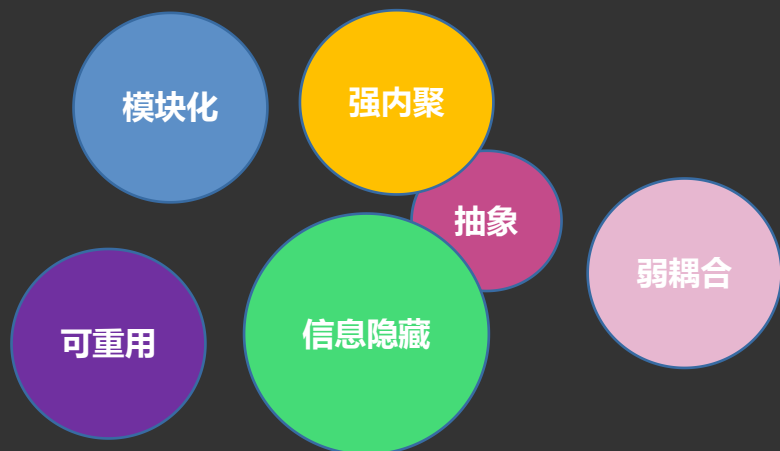
⑤ 强内聚：

内聚衡量一个模块内各个元素彼此结合的紧密程度，在设计时应该力求做到高内聚。面向对象设计的

3种内聚：服务内聚、类内聚、一般\特殊内聚

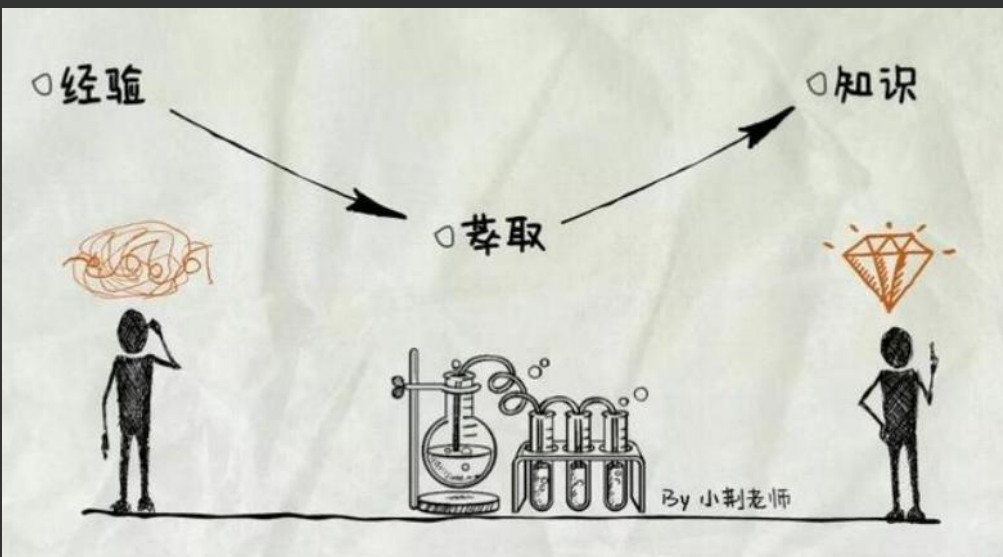
⑥ 可重用：

尽量使用已有的类，如果确实需要创建新类，则在设计这些新类的协议时，应该考虑将来的可重复使用性。



启发规则

启发式规则也称为经验法则，是在软件设计和架构中，通过长期实践总结出来的一套非正式、指导性的建议，是一种基于**经验判断**的决策技术，通过简化模型或规则快速生成可行解。



没什么好说的了！！

- ① 设计结果应该清晰易懂 (**看不懂就不会重用**)
- ② 一般-特殊结构的深度应适当 (**查族谱好麻烦，导致认知负荷极高**)
- ③ 设计简单的类 (**不能既要、又要、还要**)
- ④ 使用简单的协议 (**牵一发而动全身是不行的**)
- ⑤ 使用简单的服务 (**想苟，只提供有限服务**)
- ⑥ 把设计变动减至最小

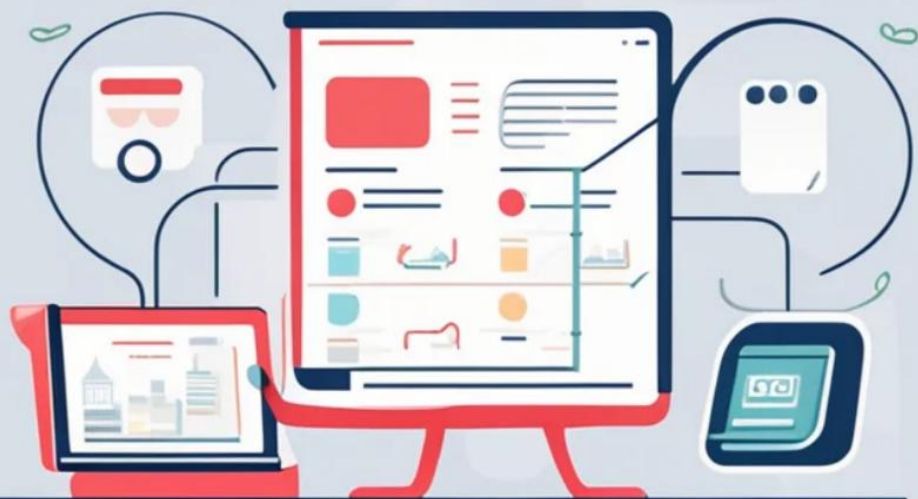


嘻嘻 我懂了

**广大人民群众看得懂的设计
才是最好的设计！！**

软件重用

重用是指同一事物不作修改或稍加改动就多次重复使用，**软件重用** (SoftWare Reuse) 是一种通过**重复使用现有软件成分**构建新系统的计算机工程方法。



广义地说，软件重用可分为以下**3个层次**：

- ① 知识重用
- ② 方法和标准的重用
- ③ **软件成分的重用**

① 软件成分的重用级别

代码重用	- 源代码片段复用 - 函数/类复用 - 库/模块复用 - 组件/框架复用 - 服务/API复用 - 设计模式与架构模式复用
设计结果重用	
分析结果重用	

② 典型的可重用软件成分

- 项目计划 - 成本估计 - 体系结构 - 需求模型和规格说明 - 设计	- 源代码 - 用户文档和技术文档 - 用户界面 - 数据 - 测试用例
---	--

类构件：面向对象技术中的“类”，是比较理想的可重用软构件，不妨称之为**类构件**。

- 实例重用**：实例重用指的是：不创建新的对象，而是使用一个已经存在的、合适的对象来完成工作。它的核心目的是避免不必要的对象创建开销（节省内存和CPU），或者是为了共享状态。

```
// 糟糕的方式：每次打印都创建新打印机实例
class BadPrinterManager {
    public void printDocument(String document) {
        // 每次调用都创建一个全新的Printer对象
        Printer printer = new Printer(); // 昂贵操作!
        printer.print(document);
        // 方法结束，printer对象被垃圾回收，巨大的浪费!
    }
}

// 客户端代码
BadPrinterManager manager = new BadPrinterManager();
manager.printDocument("Report1.pdf"); // 创建Printer1，用完丢弃
manager.printDocument("Report2.pdf"); // 创建Printer2，用完丢弃
manager.printDocument("Report3.pdf"); // 创建Printer3，用完丢弃
```

```
// 好的方式：重用同一个打印机实例
class GoodPrinterManager {
    // 在类内部持有一个（或少量）共享的打印机实例
    private Printer printer; // 声明一个引用

    // 在构造方法中初始化这个共享实例
    public GoodPrinterManager() {
        this.printer = new Printer(); // 昂贵的初始化只发生一次!
    }

    // 所有打印请求都交给这个共享的printer实例来完成
    public void printDocument(String document) {
        printer.print(document); // 重用已有的实例
    }
}
```


• 继承重用

继承重用：创建一个新的类（派生类/子类）来继承一个已存在的类（基类/父类），从而直接获得父类的属性和方法，无需重新编写它们。它的核心是“是一个 (Is-a)”的关系和代码复用。

- a. 直接重用父类的代码（字段和方法）。
- b. 扩展父类的功能（添加新的字段和方法）。
- c. 修改父类的行为（重写/Override 父类的方法）

```
java
public class Shape {
    // 共有的属性
    protected String name;

    // 构造方法
    public Shape(String name) {
        this.name = name;
    }

    // 共有的方法（这里先提供一个默认实现，或者设为抽象方法）
    public double calculateArea() {
        System.out.println("图形的面积计算逻辑未定义。");
        return 0;
    }

    public double calculatePerimeter() {
        System.out.println("图形的周长计算逻辑未定义。");
        return 0;
    }

    // 一个已经实现好的、可被重用的方法
    public void displayInfo() {
        System.out.println("这个图形的名称是: " + name);
    }
}
```

```
java
// 另一个派生类 - 同样继承自Shape
public class Circle extends Shape {
    // 自己特有的属性
    private double radius;

    public Circle(String name, double radius) {
        super(name); // 重用父类构造方法
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        // 提供圆形面积的计算实现
        return Math.PI * radius * radius;
    }

    @Override
    public double calculatePerimeter() {
        // 提供圆形周长的计算实现
        return 2 * Math.PI * radius;
    }

    // 同样直接重用父类的 displayInfo() 方法
}
```

• 多态重用

多态重用：编写一段通用的、基于父类/接口的代码，这段代码不需要修改，就能自动适用于所有子类对象。当你添加新的子类时，这段通用代码依然能正常工作。这就实现了代码的“重用”——通用逻辑只写一次，多处适用。

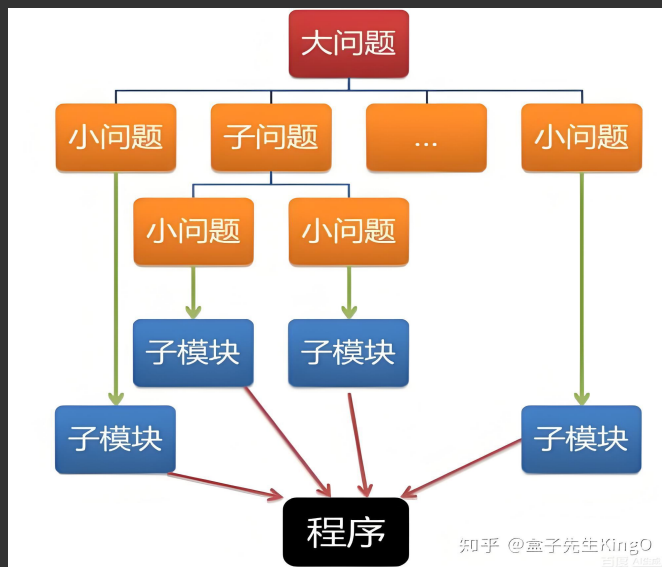
```
java
// 1. 创建对象
Shape circle = new Circle();
Shape rectangle = new Rectangle();
Shape triangle = new Triangle();

// 2. 关键点：将它们全部传递给同一个通用方法 drawShape
System.out.println("---- 逐个绘制 ----");
DrawingTool.drawShape(circle);    // 输出：正在绘制一个圆形 ○
DrawingTool.drawShape(rectangle); // 输出：正在绘制一个矩形 □
DrawingTool.drawShape(triangle);  // 输出：正在绘制一个三角形 △

// 3. 也可以把它们放进一个数组，用另一个通用方法批量处理
System.out.println("---- 批量绘制 ----");
Shape[] myArtwork = {circle, rectangle, triangle, new Circle()};
DrawingTool.drawAllShapes(myArtwork);
// 输出：
// 正在绘制一个圆形 ○
// 正在绘制一个矩形 □
// 正在绘制一个三角形 △
// 正在绘制一个圆形 ○
}
```

系统分解

“分而治之，各个击破”——在设计比较复杂的应用系统时普遍采用的策略，也是首先把系统分解成若干个比较小的部分，然后再分别设计每个部分。



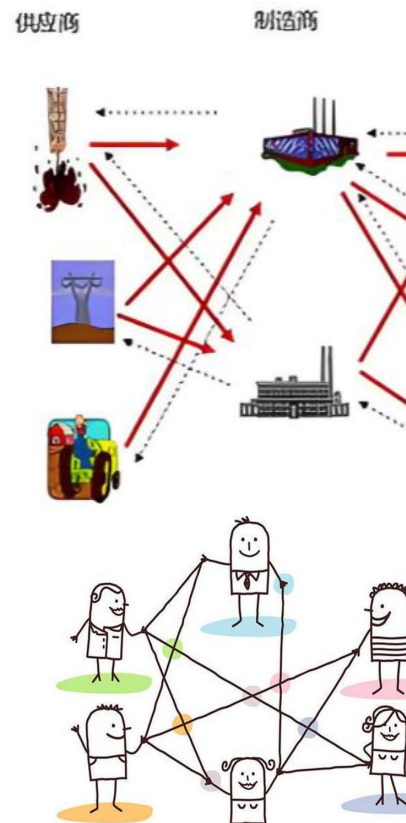
系统的主要组成部分称为**“子系统”**，子系统之间应该具有尽可能**简单明确的接口**。
应尽量减少子系统彼此间的**依赖性**。

大多数系统的面向对象设计模型，在逻辑上都由**问题域子系统**、**人机交互子系统**、**任务管理子系统**和**数据管理子系统**构成。面向对象设计模型的**4大组成部分**可以想象成整个模型的4个垂直切片。



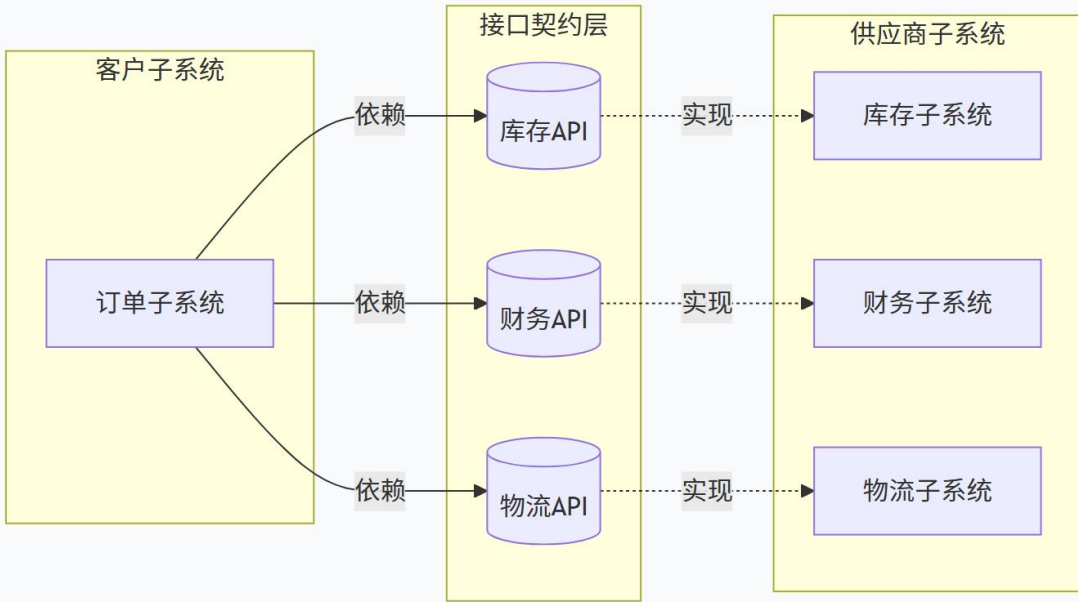
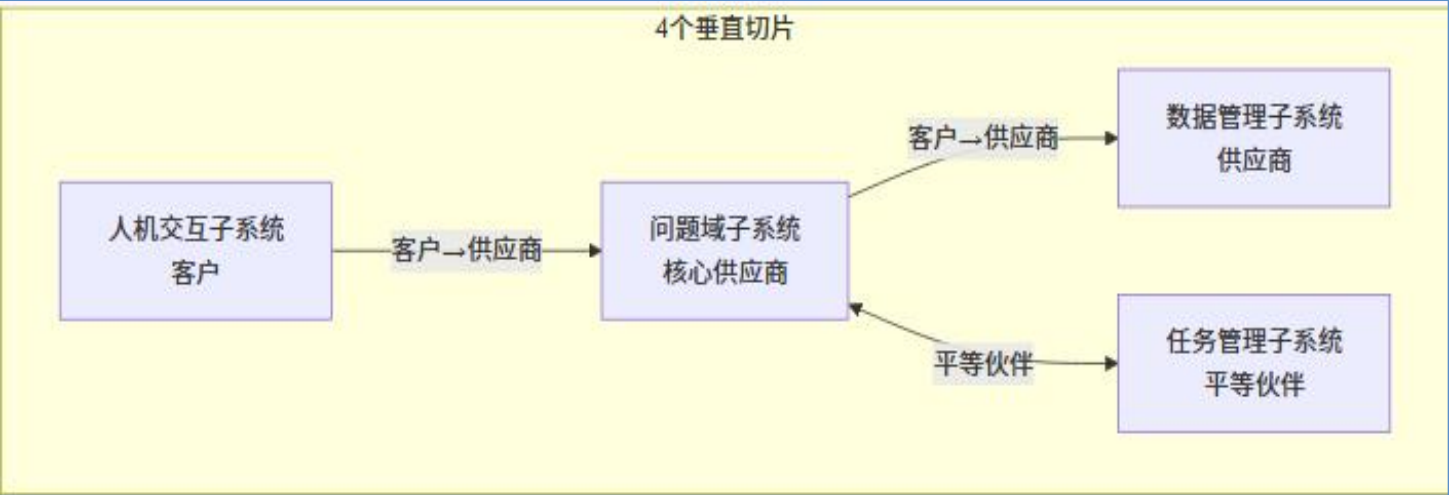
① 子系统之间的两种交互方式

- 客户-供应商关系**：作为“客户”的子系统调用作为“供应商”的子系统，后者完成某些服务工作并返回结果。前者必须了解后者的接口，然而后者却无须了解前者的接口，因为任何交互行为都是**由前者驱动的**。
- 平等伙伴关系**：每个子系统都可能调用其他子系统，每个子系统都必须了解其他子系统的接口。子系统之间的交互更复杂，这种交互方式还可能存在通信环路



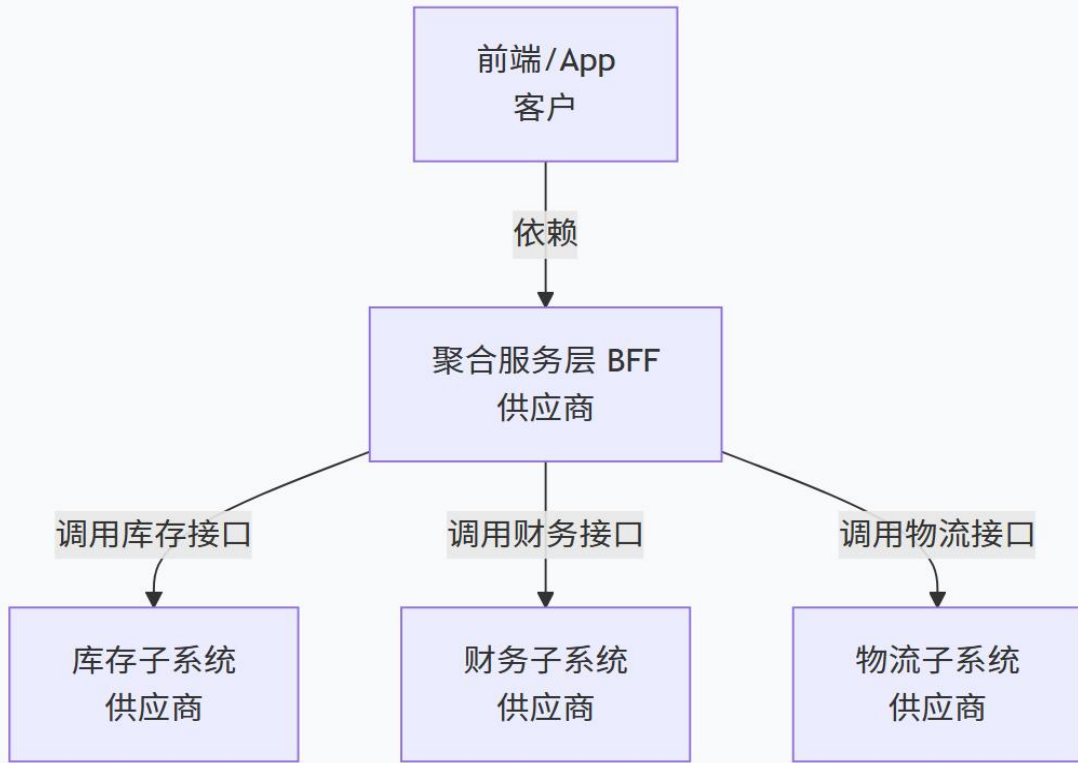
总地说来，单向交互比双向交互更容易理解，也更容易设计和修改，因此应该尽量使用**客户-供应商关系**

客户-供应商关系实例（电商系统）

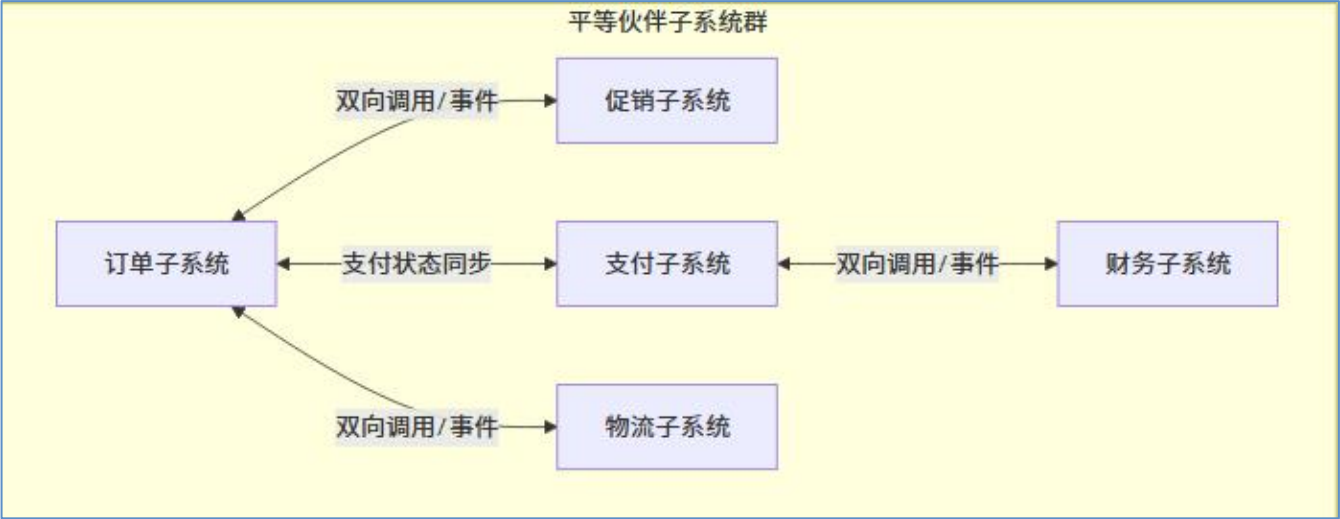
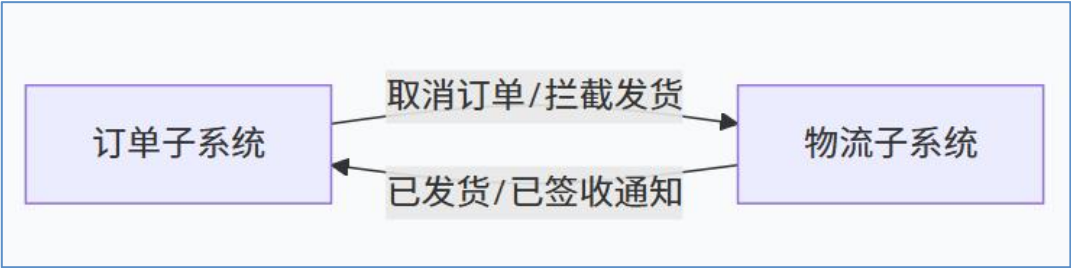
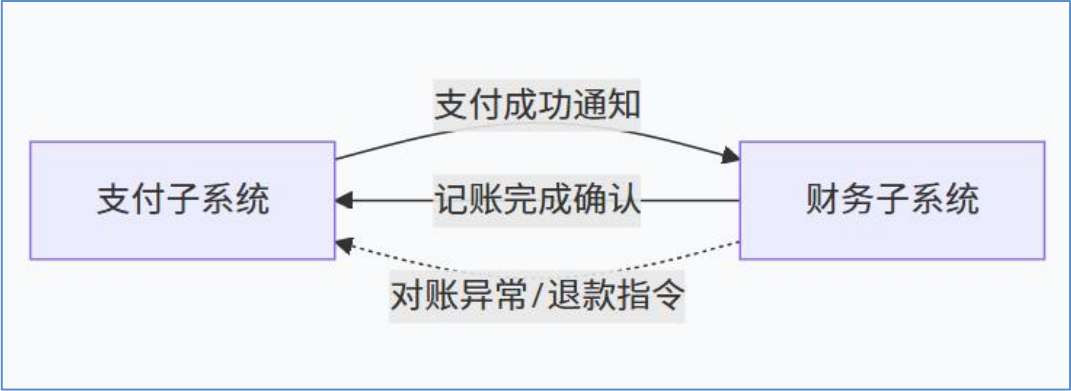
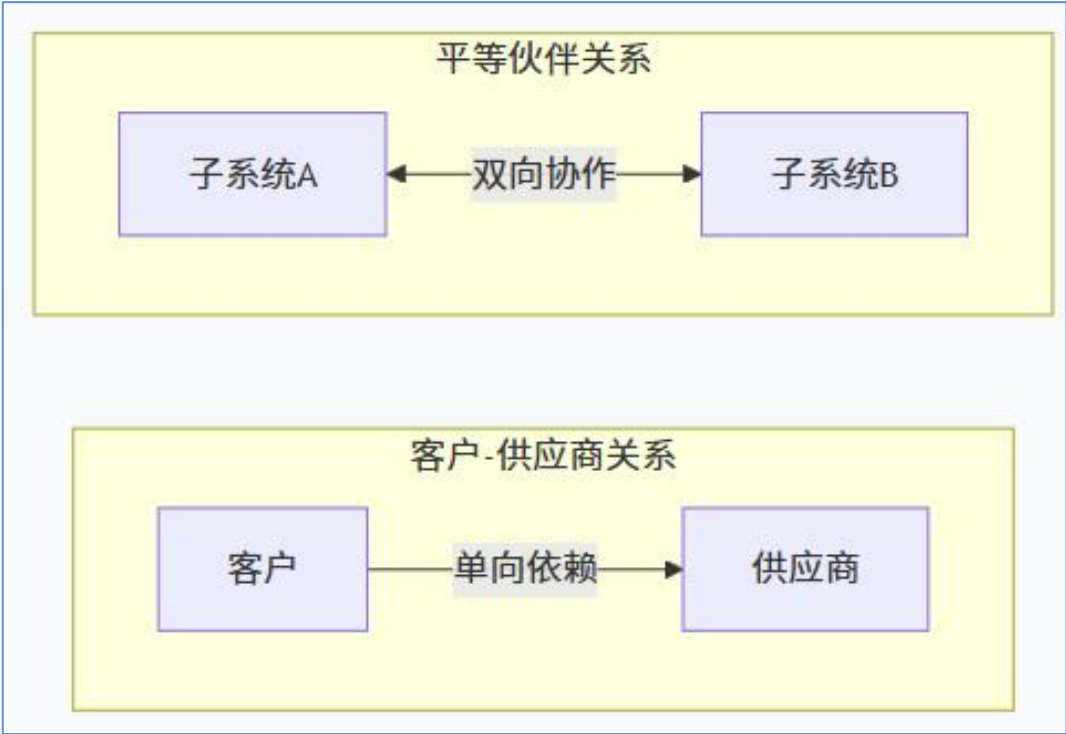
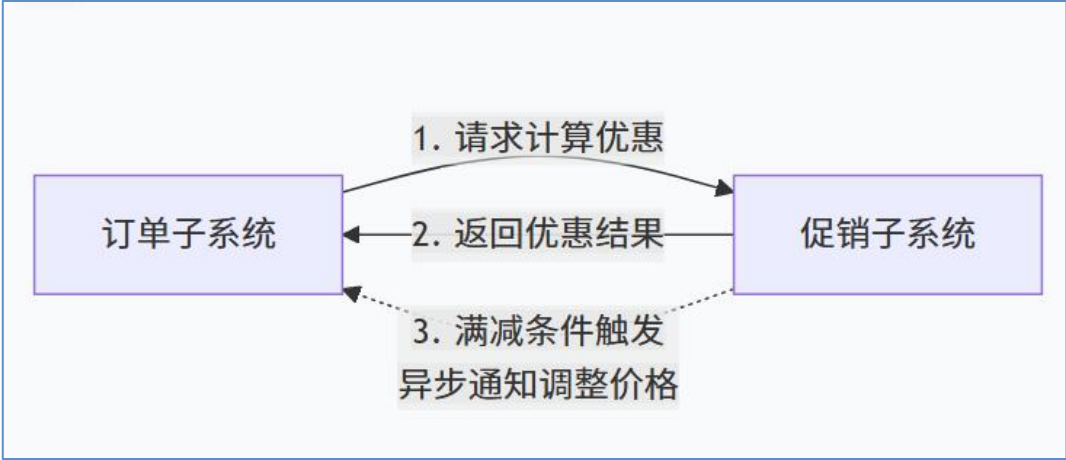


具体交互说明

交互方向	关系类型	说明	示例
人机交互 → 问题域	客户-供应商	UI调用业务服务完成用户请求	<code>PlaceOrderUI</code> 调用 <code>OrderService.placeOrder()</code>
问题域 → 数据管理	客户-供应商	业务层调用仓储接口保存/查询数据	<code>OrderService</code> 调用 <code>OrderRepository.save(order)</code>
问题域 ↔ 任务管理	平等伙伴	业务触发异步任务，任务完成后回调业务	下单后触发 <code>SendEmailTask</code> ，发送成功后记录日志



平等伙伴关系实例（电商系统）



② 组织系统的两种方案

把子系统组织成完整的系统时，有**水平层次组织**和**垂直块组织**两种方案

水平层次组织

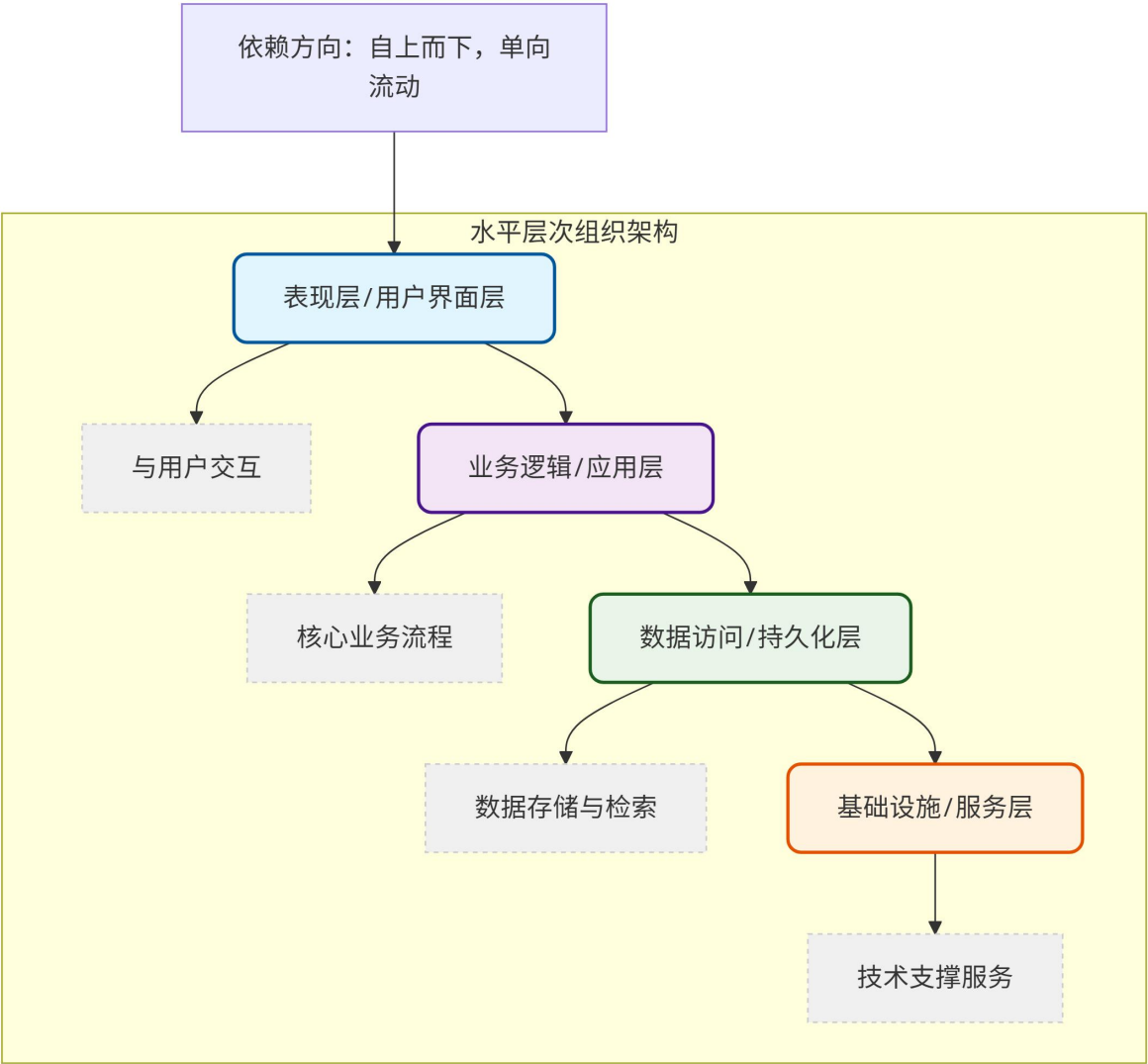
- 把软件系统组织成一个层次系统，每层是一个子系统。
- 上层在下层的基础上建立，下层为实现上层功能而提供必要的服务。
- 每一层内所包含的对象，彼此间相互独立，而处于不同层次上的对象，彼此间往往有关联。
- 在上、下层之间存在客户-供应商关系。低层相当于供应商，上层相当于客户。

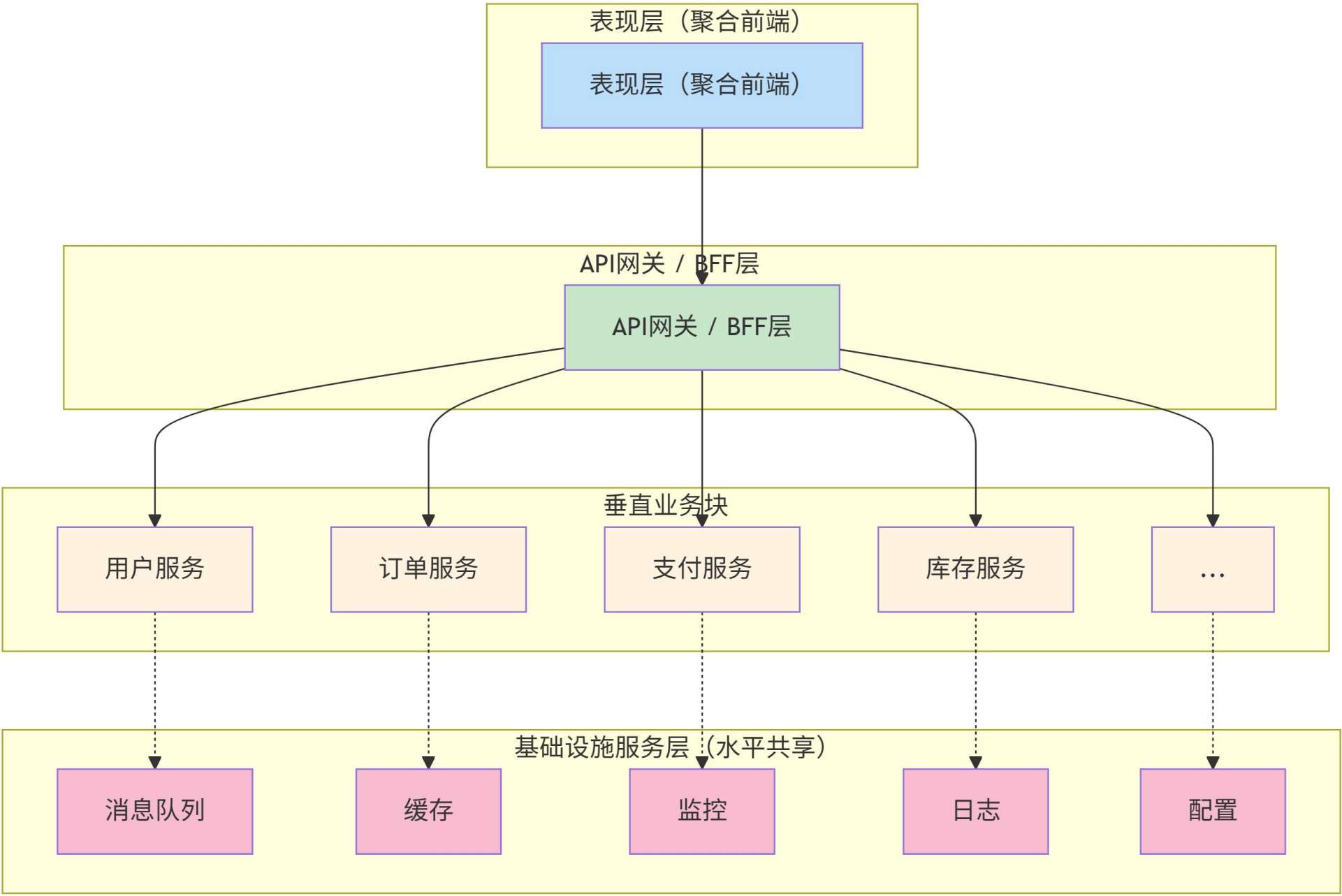
垂直块组织

- 把软件系统垂直地分解成若干个相对独立的、弱耦合的子系统。
- 一个子系统相当于一块，每块提供一种类型的服务。每个子系统负责一个完整的业务能力（功能线/用例线），而不是按技术层（如数据库层、服务层、界面层）水平切分。

现代系统通常采用**混合模式**：在业务层面垂直切分（**领域驱动**），在技术层面水平分层（**基础设施共享**），通过清晰的边界和契约保持系统的可维护性和可演进性。

应 用 软 件 包		
人机对话控制	窗口图形	仿真软件包
	屏幕图形	
	像素图形	
操 作 系 统		
计 算 机 硬 件		





java

```
// 一般化类示例
// 1. 抽象类作为一般化类
public abstract class PaymentProcessor {
    // 定义协议：所有支付处理器都必须实现的方法
    public abstract PaymentResult process(PaymentRequest request);
    public abstract boolean validate(PaymentRequest request);

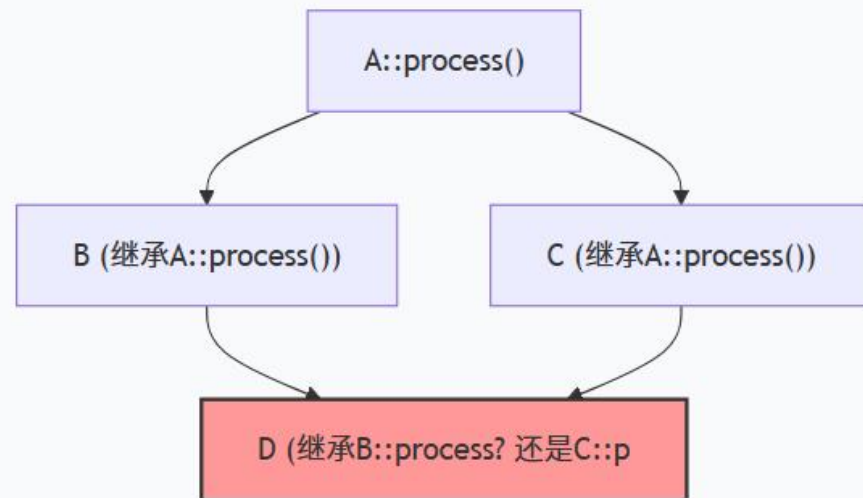
    // 可以提供通用实现
    public void logTransaction(PaymentRequest request) {
        // 通用的日志记录逻辑
    }
}

// 2. 接口作为一般化类
public interface Serializable {
    // 协议：任何可序列化的类都必须实现这些方法
    byte[] serialize();
    void deserialize(byte[] data);
}
```

分析工作可以而且应该与具体实现无关

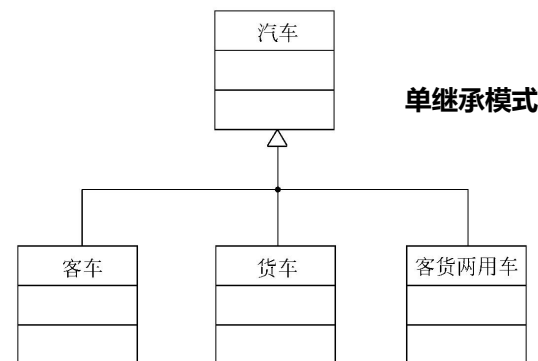
设计工作则在很大程度上受具体实现环境的约束

通过面向对象分析所得出的问题域精确模型，为设计问题域子系统奠定了良好的基础，建立了完整框架。应该保持面向对象分析所建立的问题域结构。通常，面向对象设计进需要从实现角度对问题域模型做一些补充或修改。

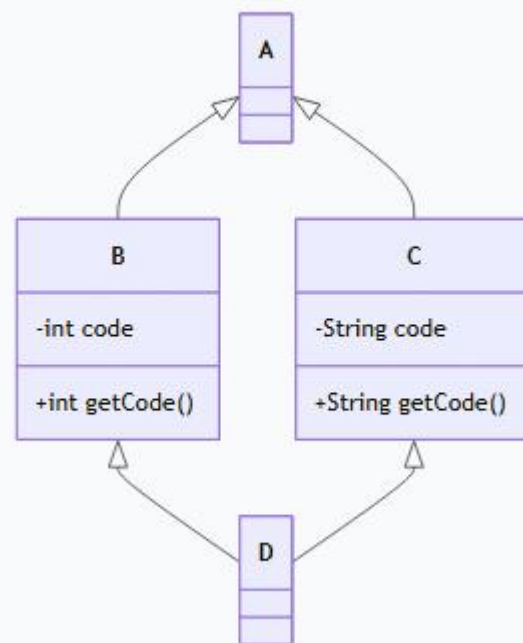
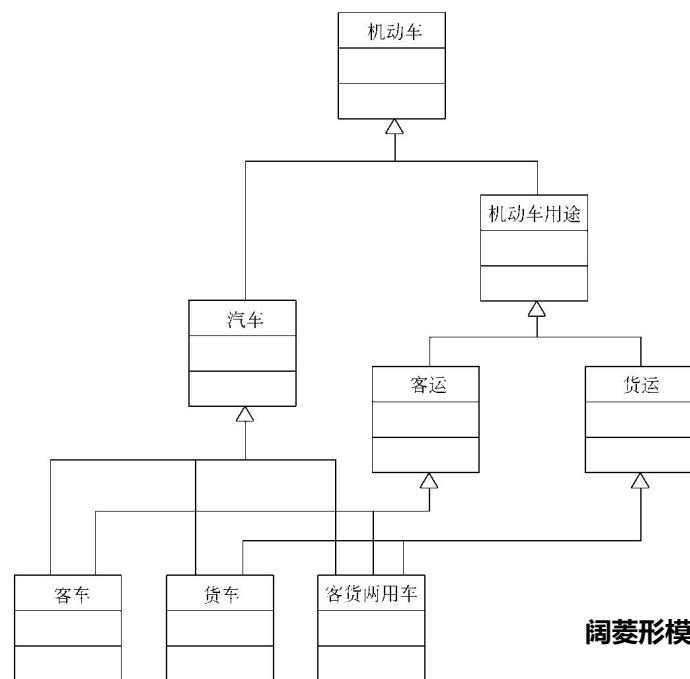


不能准确反映用户需求)

降到最低类)



⑤ 调整继承层次 (多重继承关系)



设计人机交互子系统

人机交互是一门研究系统与用户之间的交互关系的学问。

系统可以是各种各样的机器，也可以是计算机化的系统和软件。人机交互界面通常是指**用户可见的部分**。用户通过人机交互界面与系统交流，并进行操作。

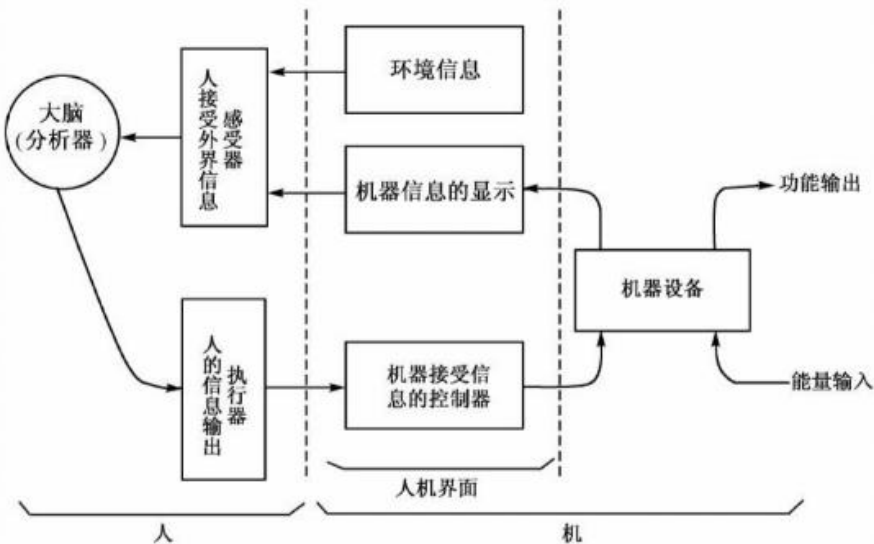


图5-1 人机信息交换系统模型

用户研究与需求分析

1. 明确使用人群（年龄、技能、文化背景等）。
2. 分析用户在什么环境下使用系统（办公、户外等）。
3. 通过访谈、问卷、观察等方法，挖掘用户真实需求。
4. 竞品分析：研究类似系统的优缺点。

交互设计

1. 交互模式选择：根据场景选择自然交互（语音、手势）或传统交互（键鼠、触摸）。
2. 原型设计：制作低保真线框图或高保真可交互原型。
3. 反馈与响应：确保用户操作后系统有即时反馈（如动画、声音）。

原型测试与迭代

1. 可用性测试：邀请真实用户测试原型，观察操作难点。
2. A/B测试：对比不同设计方案的效率。
3. 数据驱动优化：通过用户行为数据（如点击热图）持续改进。

概念设计与信息架构

1. 核心功能定义：确定系统必须支持的关键任务。
2. 信息架构：组织内容与功能，设计清晰的导航和菜单结构（如树状、卡片式）。
3. 交互流程设计：绘制任务流程图，确保操作路径简洁。

界面设计 (UI)

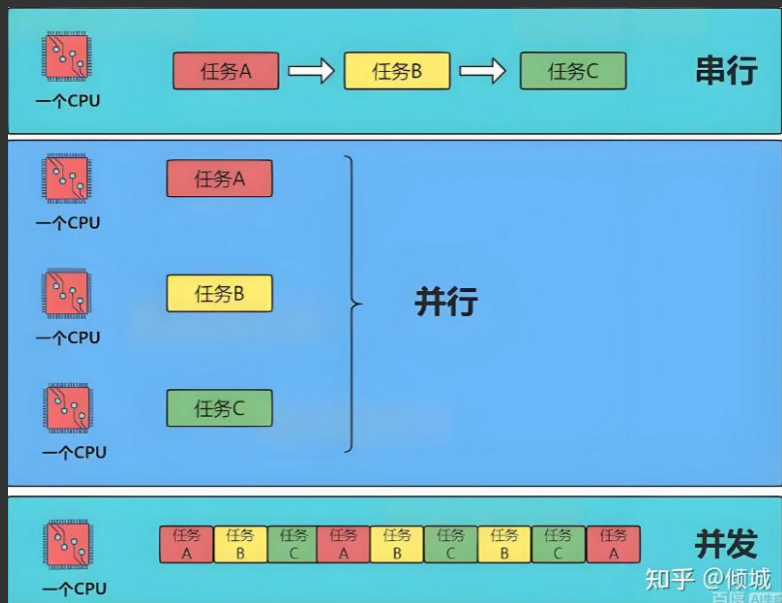
1. 视觉层次：通过色彩、排版、对比度突出重要信息。
2. 可访问性：考虑色盲、视力障碍用户。
3. 响应式设计：适配不同屏幕尺寸（桌面、移动、穿戴设备）。

开发与部署

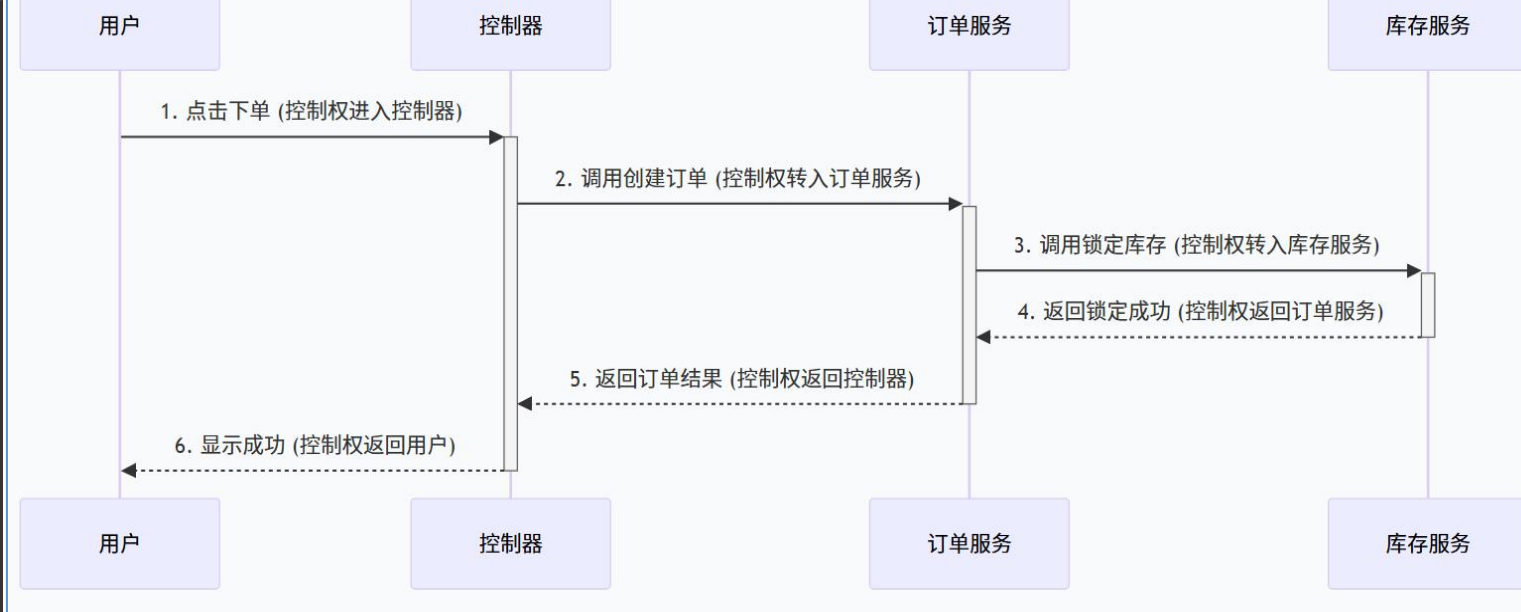
1. 技术选型：选择适合的前端框架（如React、Vue）和交互库。
2. 性能优化：确保交互响应延迟低于100毫秒（符合尼尔森准则）。
3. 多平台适配：确保跨平台体验一致性。

设计任务管理子系统

任务管理子系统是一个专门设计用于帮助组织和管理任务、项目及其相关工作的工具或软件模块。它通常是更大系统的一部分。这个子系统的核心功能包括**任务分配**、**进度追踪**、**优先级设置**、**资源管理**以及**报告生成**等。



设计工作的一项重要内容就是，确定哪些是必须同时动作的对象，哪些是相互排斥的对象。然后进一步设计任务管理子系统。



- **确定事件驱动型任务**——事件驱动
- 是一种以**事件触发**为核心的系统设计模式，指程序或系统通过收集、分发、处理事件实现任务响应的运作机制。任务的执行完全由外部或内部的特定事件触发，它的**核心逻辑**是“**等待事件→响应事件→执行任务**”。这种方式广泛应用于现代软件开发中，尤其是在用户界面、分布式系统和高并发场景中。

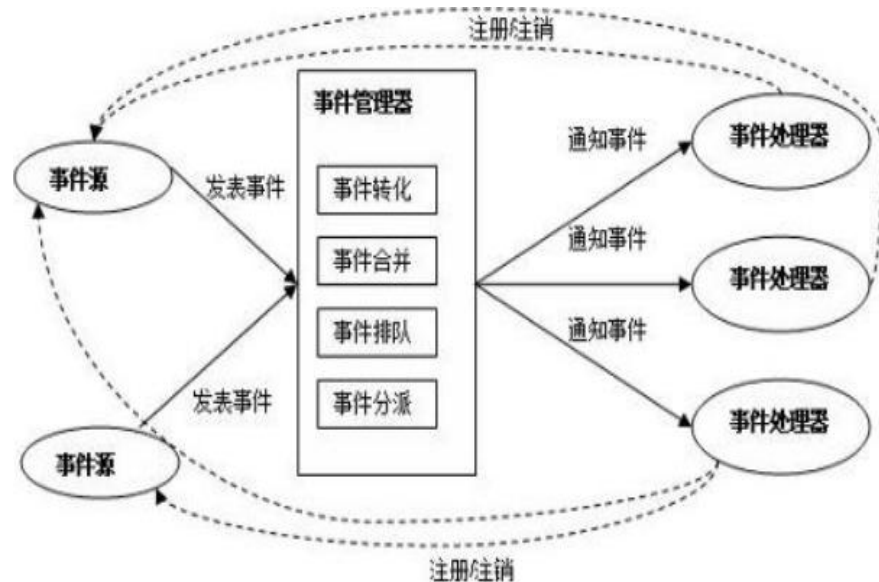


图 3.7 事件驱动式模型

设计任务管理子系统

- **确定时钟驱动型任务**——时钟驱动型任务是一种由系统时钟或时间信号触发执行的任务，它不依赖于外部事件或请求，而是按照预定的时间计划自动运行。

特性	时钟驱动型任务	事件驱动型任务
触发条件	时间	外部事件（消息、请求）
执行时机	可预测	不可预测
设计重点	调度精度、资源规划	响应速度、吞吐量
典型场景	定期维护、报表	用户请求处理、消息消费
资源使用	相对平稳	可能有突发高峰

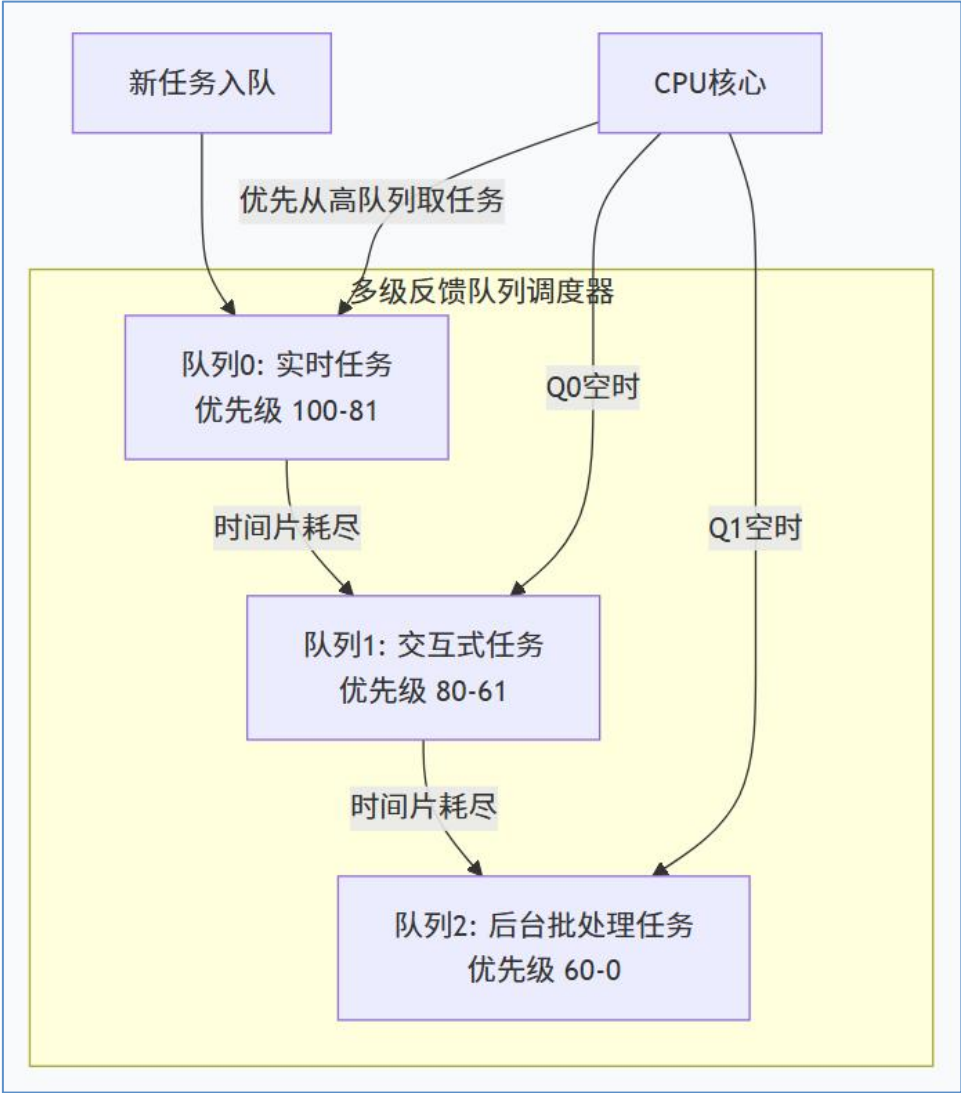
- **确定优先任务**——优先任务是指对项目目标、业务价值或时间约束具有关键影响，需要**优先分配资源、优先推进执行**的任务。

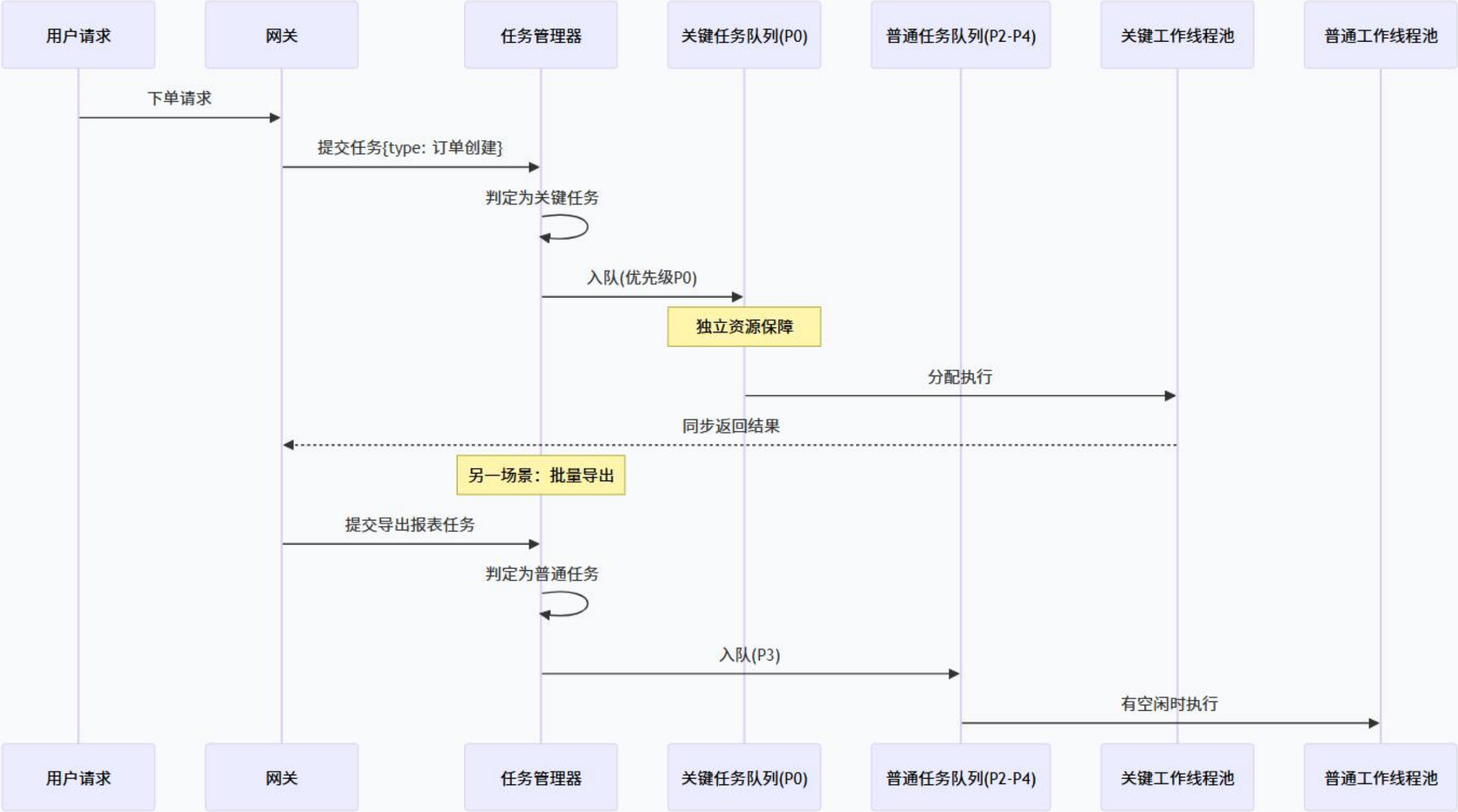
- ✓ **高优先级**：某些服务具有**很高的优先级**，为了在严格限定的时间内完成这种服务，可能需要把这类服务分离成**独立的任务**。
- ✓ **低优先级**：与高优先级相反，有些服务是低优先级的，属于低优先级处理（通常指那些背景处理）。设计时可能用额外的任务把这样的处理分离出来。

- **确定关键任务**——关键任务往往是**多个后续任务的唯一前置条件**，后续任务无法绕开它独立开展。是系统成功或失败的关键处理，这类处理通常都有严格的可靠性要求。

设计任务管理子系统

- ✓ 在设计过程中可能用额外的任务把这样的关键处理分离出来，以满足高可靠性处理的要求。
- ✓ 对高可靠性处理应该精心设计和编码，并且应该严格测试。





设计任务管理子系统

- 确定协调任务**——当系统中存在**3个以上任务**时，就应该增加一个任务，用它作为**协调任务**。是一种专门负责组织、管理和协调其他任务执行流程的特殊任务。它本身不执行具体的业务逻辑，而是作为“指挥者”或“编排者”，确保多个相关任务按照正确的顺序、依赖关系和策略执行。

与普通任务的关键区别

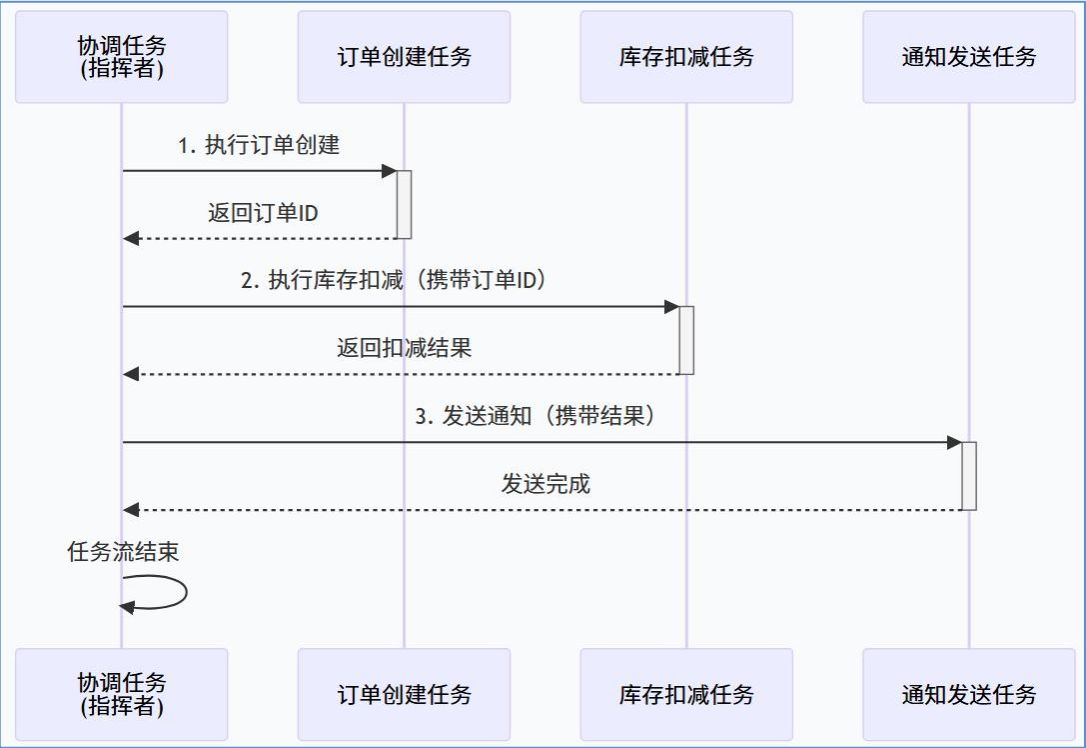
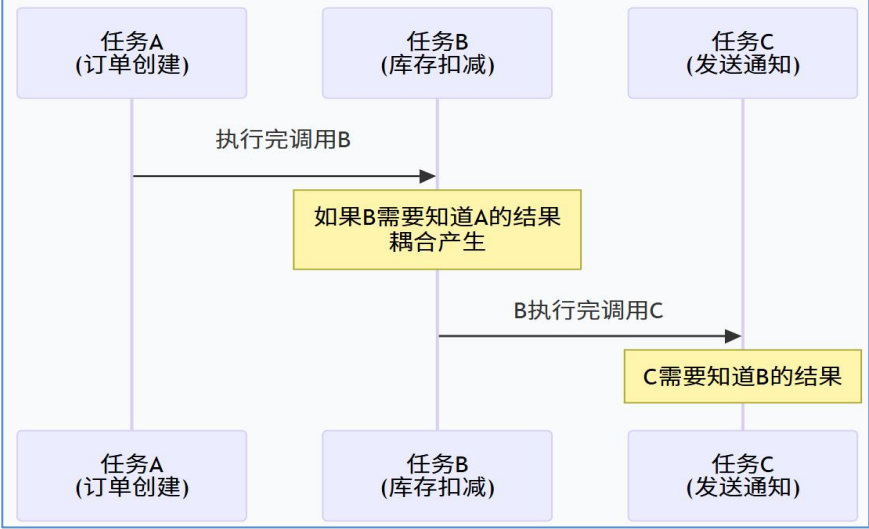
维度	协调任务	普通（工作）任务
主要职责	管理任务流程	执行业务逻辑
失败影响	整个工作流失败	仅自身失败
状态复杂度	维护多个任务状态	仅维护自身状态
执行时间	通常较长（等待其他任务）	相对较短
资源使用	内存（状态保持）、网络（协调）	CPU、I/O等

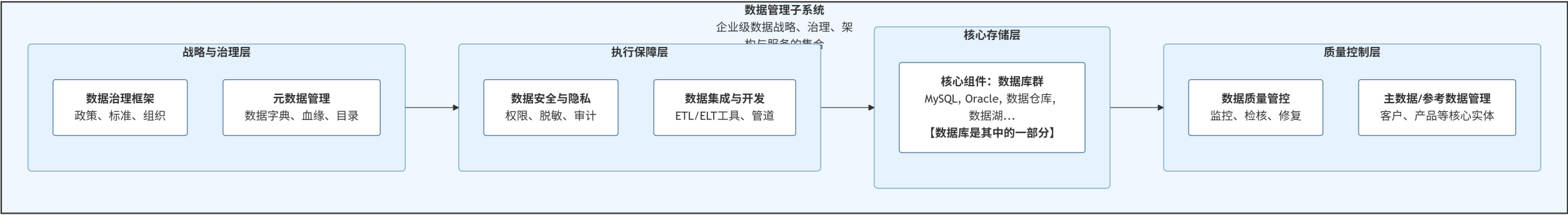
没有协调任务时，控制线是隐式散布在各任务之间，任务A、B、C互相知道对方存在，形成耦合。（右图一）

引入协调任务后，A、B、C彼此不知道对方，只与协调任务交互。控制线集中在协调任务中。（右图二）

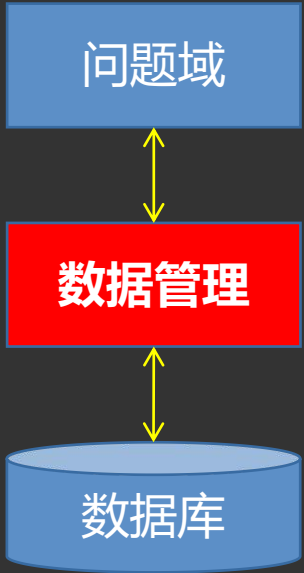
- 尽量减少任务数**——使系统中包含的任务数尽量少
- 确定资源需求**——通过计算机系统载荷估算所需要的CPU（或其他固件）的处理能力。

设计任务管理子系统





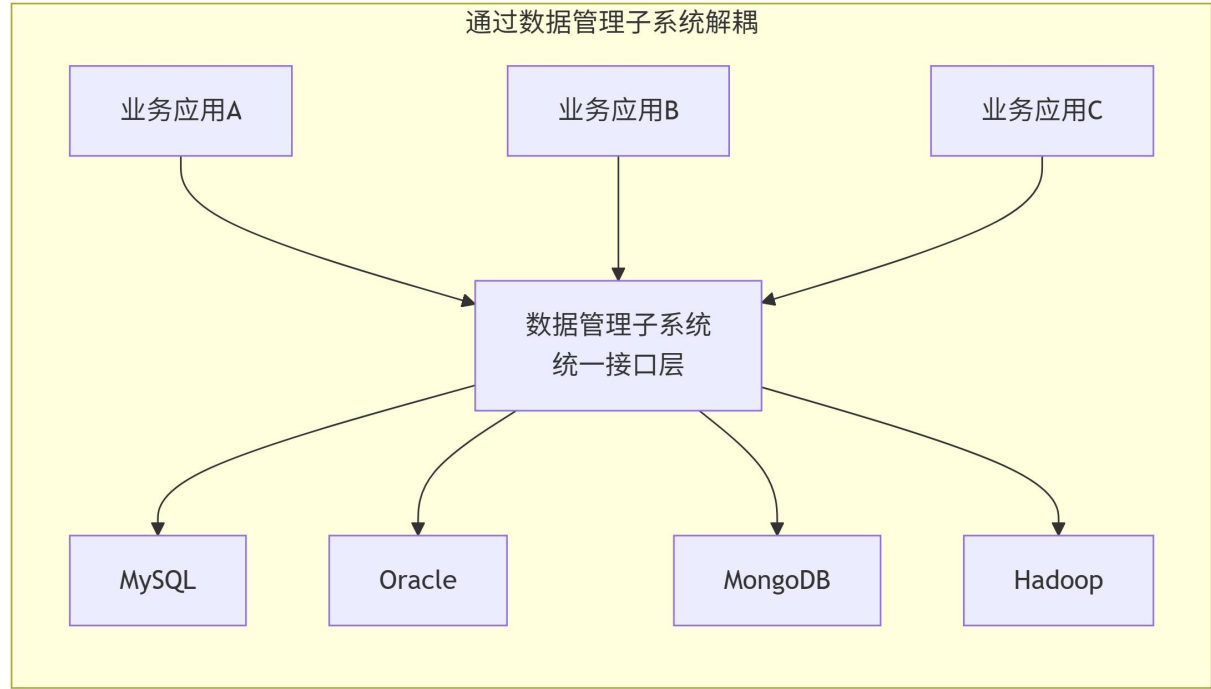
模式（文件、关系数据库、面向对象数据库）的影响。
<https://www.bilibili.com/video/BV1HP41157Hh?t=601.3>



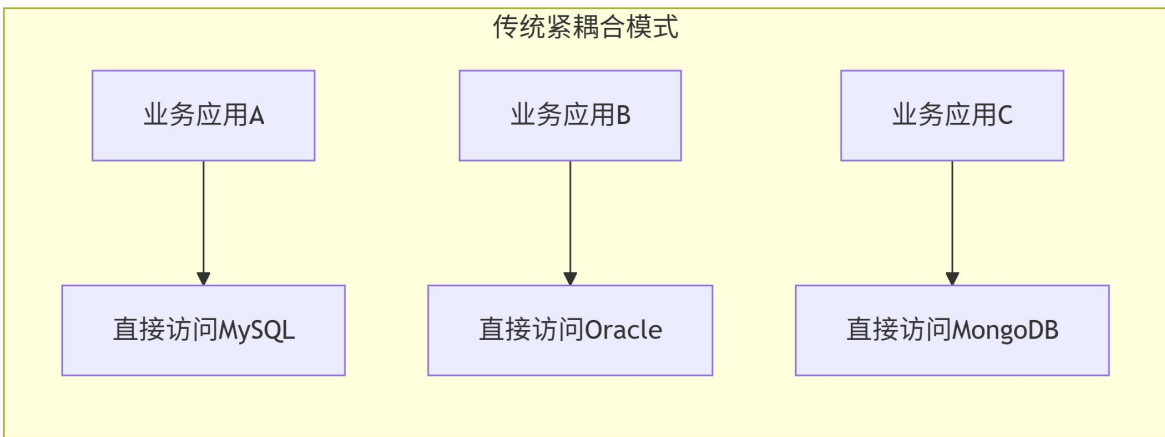
它不仅仅是数据库，而是涵盖数据从产生到归档/销毁的全流程治理体系。可以把它看作是系统的“数据中枢神经系统”。

维度	数据库	数据管理子系统
本质	一个具体的软件产品/技术。如 MySQL, Oracle, MongoDB。	一个逻辑上的系统或架构层，由一系列策略、标准、工具和流程构成。
核心职责	数据的“存、取、管”： • CRUD操作 (增删改查) • 事务管理 (ACID) • 数据存储与索引 • 查询优化与执行	数据的“控、用、护”： • 数据治理与质量 (确保数据准确、一致) • 数据安全与隐私 (权限、脱敏、合规) • 数据集成与流动 (ETL/ELT) • 元数据管理 (记录“数据的数据”) • 主数据管理 (维护核心业务实体)
范围	微观、技术性。聚焦于单个或多个数据库实例内部。	宏观、业务与技术结合。覆盖从数据产生、整合、存储到应用的全生命周期。
组成	存储引擎、查询引擎、连接器等。	包含多个数据库，以及ETL工具、数据目录、数据质量工具、数据安全平台等。
目标用户	主要是开发者、DBA（数据库管理员）。	全体数据相关角色：业务分析师、数据科学家、数据工程师、管理者、合规官等。

通过数据管理子系统解耦



传统紧耦合模式



① 选择数据存储管理模式

- **文件管理系统**——最基础的模式，以文件和目录的形式组织数据，由操作系统提供接口。
优点：简单通用，适合海量非结构化数据，成本低。
缺点：难以处理复杂查询和事务，元数据管理能力弱。
- **关系数据库管理系统**
优点：强一致性、丰富的查询能力（JOIN、事务）、技术成熟、生态完善。
缺点：水平扩展困难（分库分表复杂）、不适合海量非结构化数据。
- **面向对象数据库管理系统**（扩展的关系数据库管理系统、扩展的面向对象程序设计语言）

② 设计数据管理子系统

- 设计数据格式
文件管理系统——定义第一范式表，为每个第一范式表定义一个文件；
关系数据库管理系统——列出每个类的属性表并规范为定义第三范式表，为每个第三范式表定义一个数据库表；
面向对象数据库管理系统
- 设计相应服务

设计类中的服务

面向对象分析得出的对象模型，通常并不详细描述类中的服务。面向对象设计则是**扩充、完善和细化**面向对象分析模型的过程，设计类中的服务是它的一项重要工作内容。

定义服务：
UML描述操作的语法格式如下：
可见性 操作名（参数表）： 返回值类型(性质串)例如（银行账户取款）：

+ withdraw(amount: double = 0.0): boolean

```
classDiagram
    class BankAccount {
        - accountNumber: String
        - balance: double
        - interestRate: double
        # owner: String
        + BankAccount(accNum: String, own: String)
        + deposit(amount: double): boolean
        + withdraw(amount: double = 0.0): boolean
        + getBalance(): double
        <u>+ calculateInterest(principal: double): double</u>
        - logTransaction(message: String): void
    }
```

确定类中应有的服务需要综合考虑**对象模型、动态模型和功能模型**，才能正确确定类中应有的服务。**对象模型**是进行对象设计的基本框架。

确定类中应有的服务

状态图描绘了一类对象的生命周期，图中的状态转换是执行对象服务的结果。接收到消息的对象必然有由消息选择符指定的服务。

功能模型指明了系统必须提供的服务。状态图中状态转换所触发的动作，在功能模型中有时可能扩展成一张**数据流图**。数据流图中的某些处理可能与对象提供的服务相对应。

设计实现服务的方法

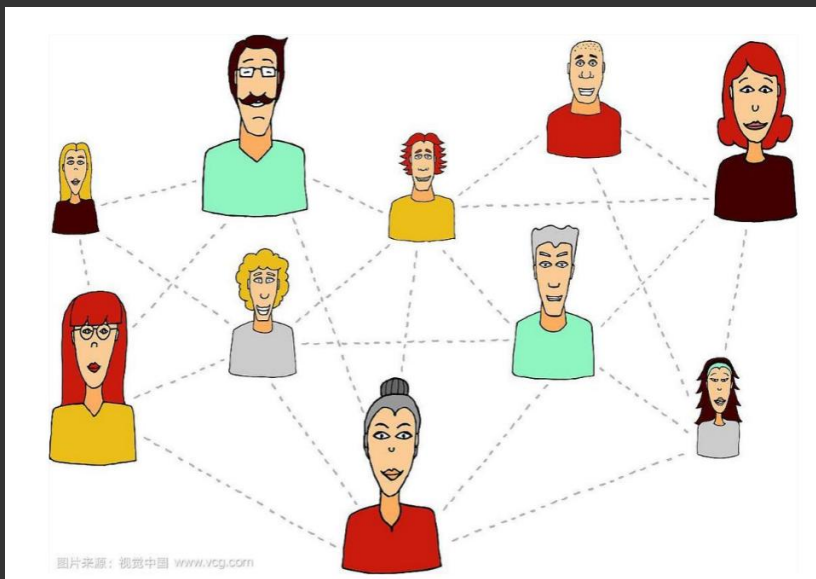
何时需要为一个类设计服务方法？

- 响应外部事件**：如用户操作、系统事件
- 改变对象状态**：状态图中的状态转换
- 与其他对象协作**：完成跨对象的业务逻辑
- 执行计算或处理**：需要复杂算法
- 访问或修改数据**：但需保持封装性
- 实现接口契约**：实现某个接口定义的行为

设计实现服务的算法	选择数据结构	算法与数据结构关系	定义内部类和 内部操作
① 算法复杂度：通常选用复杂度较低（即效率较高）的算法，但也不要过分追求高效率，应以能满足用户需求为准。 ② 容易理解与容易实现：容易理解与容易实现的要求往往与高效率有矛盾，设计者应该对这两个因素适当折衷。 ③ 易修改：应该尽可能预测将来可能做的修改，并在设计时预先做些准备。	在分析阶段，仅需考虑系统中需要的信息的逻辑结构，在面向对象设计过程中，则需要选择能够方便、有效地实现算法的物理数据结构。（例如：内存中的数据结构、数据库中的物理存储等）	① 分析问题寻找数据特点，提炼出所有可行有效的算法； ② 定义与所提炼算法相关联的数据结构； ③ 依据此数据结构进行算法的详细设计； ④ 进行一定规模的实验与评测； ⑤ 确定最佳设计。	在面向对象设计过程中，可能需要增添一些在需求陈述中没有提到的类，这些新增加的类，主要用来存放在执行算法过程中所得出的某些中间结果。

设计关联

在对象模型中，**关联是联结不同对象的纽带**，它指定了对象相互间的**访问路径**。在面向对象设计过程中，设计人员必须确定实现关联的具体策略。

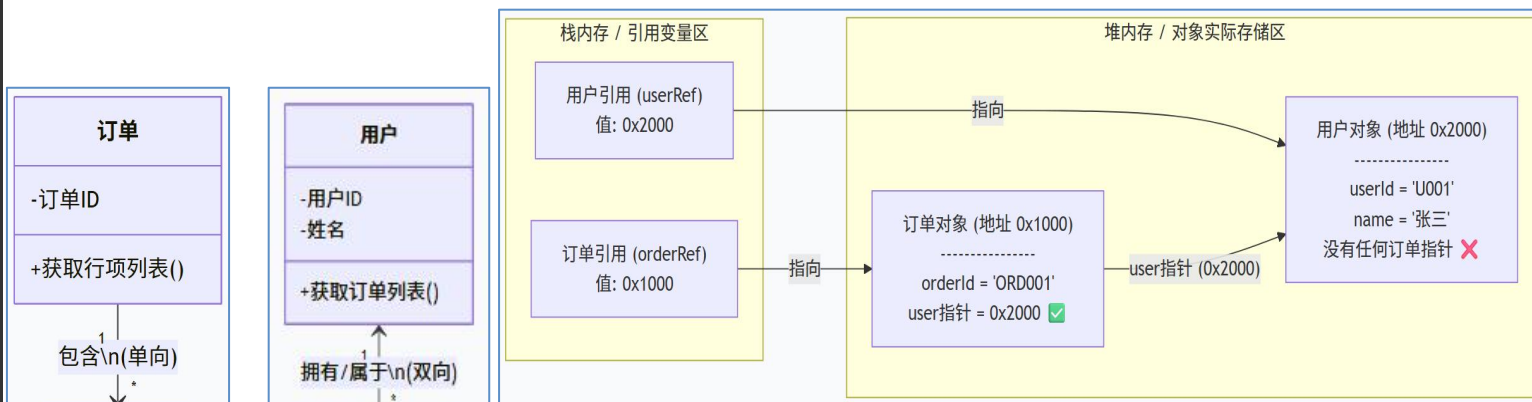


既可以选定一个全局性的策略统一实现所有关联，也可以分别为每个关联选择具体的实现策略，以与它在应用系统中的使用方式相适应。

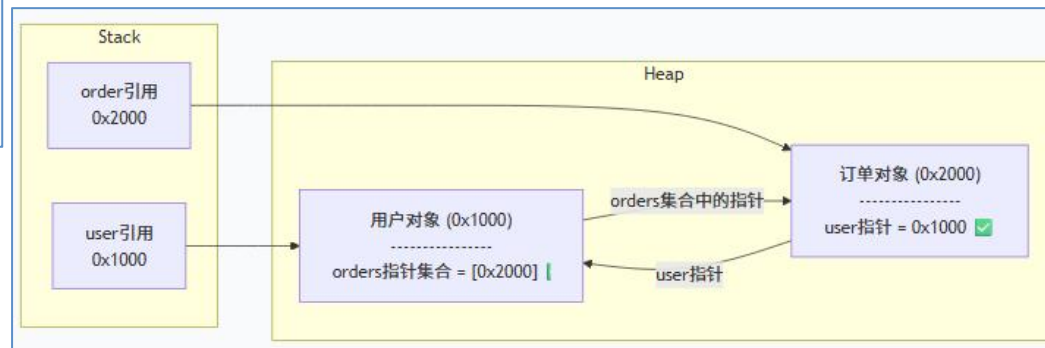
分析阶段通常倾向单向关联，设计阶段才根据具体场景决定是否引入**双向关联**。

- 应用系统使用关联有两种方式：**单向遍历**和**双向遍历**。
- 单向关联**：用一条带实心箭头的直线表示。类**A**知道类**B**的存在，并且可以访问类**B**的属性和方法；但类**B**完全不知道类**A**的存在。
- 双向关联**：一条连接两个类的实线，两端都不带箭头，或者两端都带箭头。类**A**知道类**B**的存在，类**B**也知道类**A**的存在。

通过指针（引用）能实现单向关联



双向关联实现



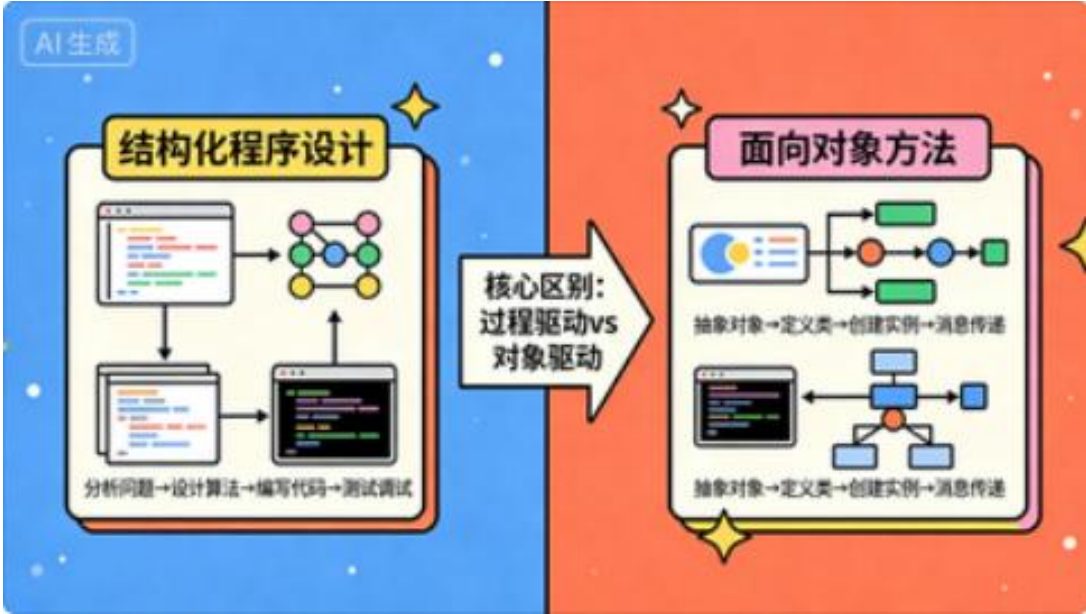
如果要想实现双向关联，需要在两端都使用指针（或指针集合），并且需要额外维护一致性。

结构化程序设计方法与面向对象方法的比较

这两种范式是软件开发史上两个里程碑，分别代表了不同时代的
核心思想，而不是简单的**优劣关系**。

结构化方法：“程序 = 数据结构 + 算法”

面向对象方法：“程序 = 对象 + 交互”



维度	结构化程序设计	面向对象程序设计
核心理念	算法为中心，自顶向下分解	对象为中心，模拟现实世界
基本单元	函数/过程	对象（包含数据和方法的类实例）
设计焦点	数据流和算法流程	对象间的交互和职责分配
主要目标	高效执行和清晰控制流	代码重用、灵活扩展和维护性
数据与操作关系	分离（数据和函数独立）	封装（数据和方法绑定在对象中）
典型语言	C, Pascal, Fortran	Java, C++, Python, C#
考虑因素	优先选择结构化	优先选择面向对象
团队技能	熟悉C/Pascal等传统语言	熟悉Java/C#/Python等现代语言
性能要求	极高，资源受限	可接受一定开销
系统规模	小型到中型	中到大型
变化频率	需求稳定，变化少	需求频繁变化
维护预期	短期使用	长期维护演进
领域特性	算法密集，数学计算	业务逻辑复杂，实体交互多

 **THANKS** 